

BLP Demand Estimation: Implementation Notes

Overview

This document details the implementation of the Berry, Levinsohn & Pakes (1995, *Econometrica*) demand estimation loop used across all five estimation routines (`estimation_{1,3,4,5,6}_demand_2_loop.py`). The codebase follows the computational best practices of Conlon & Gortmaker (2020, *RAND J. Econ.*) and adapts the structural model of Egan, Hortaçsu & Matvos (2017, *AER*) to the Brazilian deposit market.

All five loop scripts share an identical core architecture. The only differences are:

- **Estimation 1:** Baseline BLP (sleepiness from OLS/IV linear/IV Hausman).
- **Estimation 3:** Robustness (B-firms only).
- **Estimation 4:** Robustness (pooled B and D-firms, linear link).
- **Estimation 5:** Robustness (pooled B and D-firms, logistic link for ϕ).
- **Estimation 6:** Robustness (cooperative/state-owned controls, Alt-2 linear/logistic).

Each loop script loads **pre-merged Parquet files** produced by the corresponding `estimation_X_demand_1_prep.py` scripts, which contain the long panel with pre-computed sleepiness-adjusted shares.

1. The Outside Option

1.1 Conceptual Role

In BLP, the outside option $j = 0$ represents the choice of "not depositing" (or equivalently, choosing an unmodeled savings vehicle). The utility of the outside option is normalized to zero: $V_{i0mt} = 0$, so that $\exp(V_{i0mt}) = 1$. This normalization is required for the multinomial logit probability to be well-defined: shares sum to strictly less than one, and the contraction mapping $T(\delta)$ is guaranteed to be a contraction (Berry 1994, *RAND*).

Without an outside option in the softmax denominator, the model degenerates into a conditional logit over the inside goods only. The contraction mapping loses its contraction property because the level of δ becomes indeterminate: shifting all δ_j by a constant c leaves shares unchanged. This "shift-invariance" causes the inner loop to drift without converging.

1.2 Implementation in Code

In `compute_model_shares()` (all loop scripts), the outside option enters the joint log-sum-exp denominator as follows:

```
# OUTSIDE_EPS = 1.0 corresponds to  $V_0 = 0 \rightarrow \exp(0) = 1.0$ 
OUTSIDE_EPS = 1.0
log_outside = np.log(OUTSIDE_EPS) # = 0.0

joint_max = np.maximum(np.maximum(log_B_sum, log_D_sum), log_outside)
log_denom = joint_max + np.log(np.maximum(1e-300,
    np.exp(log_outside - joint_max)
    + np.exp(log_B_sum - joint_max)
    + np.exp(log_D_sum - joint_max))))
```

Setting `OUTSIDE_EPS = 1.0` is equivalent to normalizing $V_0 = 0$ exactly. The `exp(log_outside - joint_max)` term in the log-sum-exp is numerically safe: when `joint_max >> 0` the outside option contribution is exponentially small; when all inside utilities are very negative, the denominator is dominated by the outside option and shares approach zero, as they should.

1.3 Why Not a Structural Market-Size Denominator?

Standard BLP implementations (e.g., Nevo 2001) define the outside option share as $s_0 = 1 - \sum_j s_j$ and use a known potential market size M_{mt} to go from observed quantities to shares ($s_j = q_j / M_{mt}$). In our implementation, the market-size calibration happens **upstream** in the demand prep scripts (`estimation_X_demand_1_prep.py`), which compute:

$$\gamma = 1.1 \times \max_{m,t} \left(\frac{\sum_j \text{Dep}_{jmt}^{\text{Act}}}{\text{pop}_{mt}} \right)$$

This γ (a per-capita potential deposit capacity) converts raw deposit balances into shares that mechanically satisfy $\sum_j s_j < 1$. B-firm shares are:

$$s_{jkmt}^B = \frac{\text{Dep}_{jkmt}^{\text{Act}}}{\gamma \cdot \text{pop}_{mt}}$$

D-firm shares are:

$$s_{jkt}^D = \frac{\text{Dep}_{jkt}^{\text{Act}}}{\gamma \cdot \text{Pop}_t^{\text{National}}}$$

Because the shares already satisfy $\sum_j s_j < 1$ by construction, the outside option is implicitly embedded. Inside the loop, `OUTSIDE_EPS = 1.0` serves as the computational anchor that pins the level of δ .

1.4 Historical Note on the ε -Anchor

In an earlier version of the codebase, `OUTSIDE_EPS` was set to a very small value (`1e-6`), intended as a "computational epsilon" to break shift-invariance without a meaningful economic outside option. The comment in `estimation_1_demand_2_loop.py` still references this:

```
# Joint log-denominator with computational outside option (ε-anchor).  
# Small ε breaks shift-invariance, guaranteeing contraction convergence  
# without materially affecting share estimates (outside share ≈ 1e-6).
```

This was subsequently updated to `OUTSIDE_EPS = 1.0` (true structural outside option) across all five loop scripts.

2. Market Structure: B-Firms vs. D-Firms

2.1 Market Definition

The model distinguishes two firm types following Egan et al.'s national-vs-local structure:

- **B-firms** (branch-based): Operate in specific local markets indexed by MCA (Minimum Comparable Area) m . A B-firm j in market m at time t competes only in (m, t) .
- **D-firms** (digital): Operate nationally. A single D-firm row per time period t appears in the data with `mca_code = '0'` and `CODMUN_IBGE = '0'`. D-firms participate in **every** local market simultaneously.

2.2 Unified Softmax Denominator

Per Eq-4 of `V_Main.tex`, consumer i in market m at time t chooses among $J_{mt} = J_{mt}^B \cup J_t^D$. The softmax denominator for computing local choice probabilities is:

$$D_{imt} = \exp(V_0) + \sum_{j \in J_{mt}^B} \exp(V_{ijmt}^B) + \sum_{j \in J_t^D} \exp(V_{ijt}^D)$$

In the code, this is computed as

`log_denom = log(exp(log_outside) + exp(log_B_sum) + exp(log_D_sum))` , using the log-sum-exp trick for numerical stability.

2.3 Implementation of the Two Share Concepts

B-firm local share (Eq-13-B):

$$s_{jkm}^B = \frac{1}{R} \sum_{r=1}^R \frac{\exp(\delta_{jkmr} + \mu_{rjkm})}{D_{irmt}}$$

Implemented as:

```
q_B = np.exp(V_B - log_denom[b_mkt_idx, :]) # (N_B, R)
s_B = q_B.mean(axis=1) # (N_B,)
```

D-firm national share (Eq-13-D):

$$s_{jkt}^D = \sum_m \frac{M_{mt}}{M_t} \cdot \frac{1}{R} \sum_r \frac{\exp(\delta_{jkt} + \mu_{rjkt})}{D_{irmt}}$$

This is the population-weighted average of local choice probabilities across all markets in time t . The population weights `pop_weights` are pre-computed:

$$w_m = \frac{\text{pop}_{mt}}{\sum_{m'} \text{pop}_{m't}}$$

and stored in the `precomp` dictionary to avoid recomputation.

2.4 Omega: Local B-Firm Residual Share (Eq-14)

$$\Omega_{mt} = 1 - \sum_{j \in J_t^D, k} s_{jkm}^{D, \text{local}}$$

This term enters the B-firm contraction update: since B-firm conditional shares are defined relative to the Non-D residual, the contraction step for B-firms is:

```

delta_new[b_mask] = delta[b_mask]
+ ln(s^{data,B}_{cond})
+ ln(Omega)
- ln(s^{model,B})

```

compared to the standard BLP update $\delta' = \delta + \ln s^{data} - \ln s^{model}$.

3. Inner Loop: Anderson-Accelerated Contraction

3.1 The Standard BLP Contraction

The standard contraction mapping (Berry 1994) is:

$$T(\delta)_j = \delta_j + \ln s_j^{data} - \ln s_j^{model}(\delta, \theta_2)$$

This is a contraction under the ℓ_∞ norm when:

1. The outside option pins the level (preventing shift-invariance).
2. θ_2 is held fixed.

3.2 Anderson Acceleration (m=5)

Rather than iterating $\delta^{h+1} = T(\delta^h)$ until convergence, we use **Anderson mixing** (Anderson 1965; Walker & Ni 2011) with depth $m = 5$. Conlon & Gortmaker (2020, Section 3.3) show this is empirically 10–50× faster than SQUAREM when the Jacobian spectral radius is near 1.

The algorithm maintains a history of the last m iterates $\{x^{h-m+1}, \dots, x^h\}$ and residuals $\{f^{h-m+1}, \dots, f^h\}$ where $f^h = T(x^h) - x^h$. At each step it solves:

$$\min_c \left\| \sum_i c_i f^{h-m+i} \right\|^2 \quad \text{s.t.} \quad \sum_i c_i = 1$$

and sets $\delta^{h+1} = \sum_i c_i (x^{h-m+i} + f^{h-m+i})$.

Implemented as:

```

dF = F_k[:, 1:] - F_k[:, [0]]          # (N, k-1)
rhs = -dF.T @ F_k[:, 0]                 # (k-1,)
A_aa = dF.T @ dF + 1e-10 * np.eye(hist_len - 1)  # Tikhonov regularization
c_bar = np.linalg.solve(A_aa, rhs)       # (k-1,)
c[0] = 1.0 - c_bar.sum()
c[1:] = c_bar
delta = (X_hist[:, :hist_len] + F_hist[:, :hist_len]) @ c

```

The $1e-10 * I$ Tikhonov term prevents singular A when consecutive residuals are nearly collinear.

3.3 Convergence Criterion

The inner loop terminates when:

$$\|T(\delta) - \delta\|_{\infty} < \tau_{\text{inner}}$$

Default: $\tau_{\text{inner}} = 10^{-12}$. On HPC runs, we set $\tau_{\text{inner}} = 10^{-12}$ with `max_inner = 1500`.

3.4 Warm-Starting

Following Conlon & Gortmaker (2020, Section 3.2), the converged δ^* from the previous outer iteration is cached and used as `delta_init` for the next call to `blp_contraction()`. This typically reduces inner iterations from ~200 to ~10–30 for small θ_2 perturbations.

```

delta_cache: dict = {}
# ... inside gmm_objective() ...
delta_init = delta_cache.get('last_delta')
delta, converged, n_iter, _ = blp_contraction(df, mu, R, ..., delta_init=delta_init)
delta_cache['last_delta'] = delta.copy()

```

3.5 Numerical Safeguards

1. **Value clamping:** $V_{jkmr} = \delta_j + \mu_{rj}$ is clipped to $[-500, 500]$ before exponentiation.
2. **Log-sum-exp stability:** All softmax denominators use the max-subtraction trick.
3. **Share floors:** `np.clip(s, 1e-15, None)` before taking logs in the contraction step.
4. **Delta clamping:** Output of each contraction step is clipped to $[-500, 500]$.
5. **Non-finite abort:** If any δ_{new} entry is NaN/Inf, the contraction returns early.

4. Outer Loop: GMM Minimization

4.1 Objective Function

The GMM objective is (Eq-A1):

$$Q(\theta_2) = G(\theta_2)'W G(\theta_2)$$

where the moment vector G is:

$$G(\theta_2) = \frac{1}{N} \sum_i \xi_i(\theta_2) \cdot h(Z_i)$$

ξ are the structural residuals from the linear IV regression $\delta = X\theta_1 + \xi$ (Eq-A5), and Z is the instrument matrix.

4.2 Sequential CG2020 Build-Up (--stage sequence)

When `--stage sequence` is passed, the script runs four nested stages sequentially, each warm-starting from the previous:

Stage	θ_2 dim	Description
logit	0	Pure logit, $\theta_2 = 0$. Direct $\delta = \ln(s^{data})$.
sigma	1	Random coefficient on spread only.
full	5	σ : spread, log_assets. Π : spread $\times$ GDP, spread $\times$ age, spread $\times$ connectivity.
extended	8	Full + product $\times$ demographic interactions.

Stage checkpoints are saved as `.pk1` files, and the next stage loads the prior θ_2^* and δ^* via the warm-start mechanism.

4.3 Theta-1 Recovery (Linear IV, Eq-A5)

Given $\delta^*(\theta_2)$, the linear parameters $\theta_1 = (\alpha_1, \alpha_2, \alpha_4, \alpha_5, \beta)$ are recovered by 2SLS:

- First stage:** Project endogenous spreads ($k = 4, 5$) on the full instrument set $H = [X, Z^{IV}]$.
- Second stage:** $\theta_1 = (X'^{* \top} X^*)^{-1} X'^{* \top} \delta$ where $X^* = [\hat{\rho}, X]$.

3. **Residuals:** $\xi = \delta - X_{\text{full}}\theta_1$ (using **original** spreads, not projected).

Standard errors use cluster-robust variance with Carter-Schnepel-Steigerwald imbalanced-cluster correction:

$$G^* = \frac{G}{1 + \text{CV}(N_g)^2}$$

where G is the number of clusters and $\text{CV}(N_g) = \sigma(N_g)/\bar{N}_g$.

4.4 Outer Optimizer

- **Default:** L-BFGS-B with bounds $[-15, 15]$ per parameter, `ftol = 1e-6`.
- **Alternative:** Nelder-Mead (gradient-free, slower but more robust for noisy objectives).
- Theta-2 SEs are computed via a numerical Jacobian of the moment function at θ_2^* , using the GMM sandwich formula.

4.5 Weighting Matrix

The initial weighting matrix is $W = (Z'Z/N)^{-1}$. The current implementation uses one-step GMM (no iterated W update).

5. Simulation Draws

5.1 Halton Quasi-Random Draws (ν)

R draws of dimension $K_{\text{TYPES}} + L_{\text{PROD}} = 12$ are generated using scrambled Halton sequences (Nevo 2001), transformed to standard normal via inverse CDF:

```
sampler = Halton(d=dim, scramble=True, seed=seed)
u = np.clip(sampler.random(n=R), 1e-6, 1 - 1e-6)
nu = stats.norm.ppf(u) # (R, 12)
```

The ν draws interact with Σ to generate the random-coefficient heterogeneity in μ .

5.2 Demographic Draws (d)

For each (m, t) market, R demographic vectors are drawn from $\mathcal{N}(\mu_m, (0.1 \cdot \sigma_{\text{national}})^2)$ where μ_m is the observed MCA-level mean of each demographic variable and σ_{national} is the cross-MCA standard deviation.

These demographic draws interact with the Π matrix to generate the observed-heterogeneity component of μ .

5.3 R Parameter

Setting	R value	Notes
Local testing	100	CG2020: sufficient for debugging and directional accuracy.
Sigma stage (sequence mode)	100	Automatically overridden.
HPC production	1000	Final results for publication.

Memory allocation scales as $O(N \times R)$ for the μ matrix. At $R = 1000$ with $N \approx 500,000$, each (N, R) float64 matrix consumes ~3.7 GB. The SLURM scripts request 750 GB to accommodate 8 parallel workers.

6. HPC Execution

6.1 Parallel Architecture

```
submit_blp_hpc_X.sh
└─ estimation_X_demand_2_loop.py --spec 12 --stage sequence --R 1000 --workers 8
    └─ ProcessPoolExecutor(max_workers=8)
        └─ worker_blp(spec=1, stage=logit)
        └─ worker_blp(spec=2, stage=logit)
        └─ ...
        └─ worker_blp(spec=12, stage=logit)
        [then stage=sigma, then full, then extended]
```

Each spec runs in an independent process. The `--stage` sequence flag causes the four stages to execute sequentially for each spec, with warm-starting between stages.

6.2 SLURM Configuration

All scripts are submitted via identical SLURM parameters:

Parameter	Value	Rationale
<code>--partition</code>	week	Up to 7-day wall time.
<code>--time</code>	48:00:00	Conservative 2-day cap.
<code>--cpus-per-task</code>	12	8 workers + 4 overhead (GC, numpy BLAS).
<code>--mem</code>	750G	Each worker allocates (N, R) float64 matrices at $R = 1000$.
<code>--tol-inner</code>	1e-12	Publication-grade contraction tolerance.
<code>--max-inner</code>	1500	Safety cap (Anderson typically converges in <200).

6.3 Monitoring

- A background daemon thread sends hourly email digests (07:00–21:00) via the Yale SMTP relay.
- A `blp_progress.log` file is appended to the output directory with timestamped outer-iteration status.
- SLURM's `--mail-type=ALL` provides BEGIN/END/FAIL notifications.

7. Product Characteristics and Instruments

7.1 Utility Specification

The indirect utility of consumer i for product (j, k) in market (m, t) is:

$$V_{ijkmt} = \underbrace{\alpha_k \cdot \rho_{jkmt} + X'_{jmt}\beta}_{\delta_{jkmt}} + \underbrace{\nu'_i \Sigma \cdot \text{prod}_{jkmt} + d'_{imt} \Pi \cdot \text{prod}_{jkmt}}_{\mu_{irjkmt}}$$

where ρ_{jkmt} is the deposit spread (in basis points), X_{jmt} are non-price product characteristics, ν_i are i.i.d. draws, and d_{imt} are demographic draws.

7.2 Product Characteristics (X_COLS)

Variable	Description
fgc_covered	FGC deposit insurance coverage dummy
has_ip	Payment institution indicator
seg_S2 – seg_S5	Prudential segment dummies (S1 is baseline)
log_total_assets_lag	Log of lagged total consolidated assets
equity_ratio_lag	Lagged equity-to-assets ratio

7.3 Instruments

Group	Variables	Purpose
BLP- LOO	loo_log_assets , mean_loo_log_assets , ... (11 vars)	Leave-one-out means of rival characteristics (BLP 1995)
Cost- shifters	personnel_cost_ratio_lag , admin_cost_ratio_lag , tax_cost_ratio_lag	Supply-side cost instruments for endogenous spreads ($k = 4, 5$)
Capital	indice_basileia_lag	Basel III capital adequacy ratio

7.4 Demographics (D_COLS)

Variable	Description
gdp_per_capita	Municipal GDP per capita (scaled ÷ 10,000)
fraction_65plus	Share of population aged 65+
fraction_young	Share of population aged 18–29
pix_users_pf_per1000	PIX adoption density (scaled ÷ 100)
connections_per100	ANATEL mobile broadband connections (scaled ÷ 100)
frac_4g5g	4G/5G coverage fraction
branches_per1000	Bank branches per 1,000 population
cadunico_families_per1000	CadÚnico low-income families (scaled ÷ 100)

8. Output Structure

Each loop script writes per-spec results to:

```
.../ESTIMATION_OUTPUT/BLP_RESULTS/blp_results_spec_{id}_{stage}.pkl
```

Each pickle contains:

Key	Type	Description
theta1	ndarray (12,)	Linear parameters: $[\alpha_1, \alpha_2, \alpha_4, \alpha_5, \beta_1, \dots, \beta_8]$
theta1_se	ndarray (12,)	Cluster-robust standard errors for θ_1
theta2	ndarray (n_02,)	Nonlinear parameters: $[\sigma_1, \dots, \pi_1, \dots]$
theta2_se	ndarray (n_02,)	GMM sandwich standard errors for θ_2
delta	ndarray (N,)	Converged mean utilities
xi	ndarray (N,)	Structural residuals
Q_value	float	GMM objective at optimum
converged	bool	Outer optimizer convergence flag

A JSON summary (`blp_summary_{stage}.json`) is also written with abbreviated results for quick inspection.

References

- Anderson, D.G. (1965). "Iterative procedures for nonlinear integral equations." *J. ACM*, 12(4).
- Berry, S. (1994). "Estimating discrete-choice models of product differentiation." *RAND J. Econ.*
- Berry, S., Levinsohn, J. & Pakes, A. (1995). "Automobile Prices in Market Equilibrium." *Econometrica*, 63(4).
- Conlon, C. & Gortmaker, J. (2020). "Best Practices for Differentiated Products Demand Estimation with PyBLP." *RAND J. Econ.*, 51(4).

- Egan, M., Hortaçsu, A. & Matvos, G. (2017). "Deposit Competition and Financial Fragility." *AER*, 107(1).
- Nevo, A. (2001). "A Practitioner's Guide to Estimation of Random-Coefficients Logit Models." *J. Econ. & Management Strategy*, 9(4).
- Walker, H.F. & Ni, P. (2011). "Anderson Acceleration for Fixed-Point Iterations." *SIAM J. Numer. Anal.*, 49(4).